Lean2LLM: Rechecked Proof Search and Portable Proof Storage for Lean

Octavia Sulea
octavia@octav.ai
https://github.com/mary-octavia/lean-llm

22 October 2026
Software and systems manuscript; draft for author review

Abstract

Language models can propose formal proofs, while proof assistants check the resulting proof terms. A useful research workflow must also preserve the exact statement being proved, make verification conditions inspectable, and retain artifacts that another user can find and check. Lean2LLM develops these concerns around an existing local Lean–LLM bridge. Its proposal loop fixes the context and proposition, supplies Lean's actual goal and diagnostics to an external proposer, and accepts an exported proof only after Lean checking and a transitive axiom audit. An exact-request cache rechecks every hit. A separate content-addressed proof library adds indexed retrieval, explicit toolchain identities, portable bundles, and local verification records. A browser workbench checks restricted core Lean source in a fresh WebAssembly worker and exchanges records with the Python library. Retrieval proposes an artifact for use; a fresh check establishes its status in the receiving environment. A second verification profile accommodates standalone kernel certificates, drawing on a published Navier–Stokes reproducibility bundle. This paper documents the architecture, artifact schema, trust boundaries, and reproduction procedure. It reports inherited repository evidence separately from tests of the new storage implementation. The contribution is a practical composition of proof generation, checking, storage, and retrieval.

1 A proof should remain usable after the chat

An interactive proof experiment often ends with a successful compiler message and a source file. Reusing that result later requires more information: the fixed proposition, definitions in scope, proof body, permitted assumptions, Lean version, dependency identity, and the procedure that produced the verification result. A plausible explanation or a remembered success message does little to resolve a mismatch in any of these fields.

Lean2LLM joins a bounded proof-proposal loop to an artifact library. The starting point is the public `lean-llm` repository, whose native Lean tactic and Python driver already implement proposal, feedback, kernel checking, export, and exact cache replay [4]. The new library makes the stored proof an explicit exchange object. A web interface exposes the same distinction between draft source, a recorded verification event, and a check performed in the current environment. The public-facing project name is Lean2LLM; the associated domains are `https://lean2llm.com`, `https://lean4llm.com`, and `https://langlean.si`. Domain names identify interfaces, while repository revisions and artifact digests identify reproducible material.

There are three engineering objectives. First, the model must solve the given goal under its fixed statement. Second, retrieval must make an existing proof easy to discover without treating textual similarity as logical equivalence. Third, a reader should be able to download enough evidence to understand and repeat the relevant check. These objectives lead to two verification profiles and two complementary forms of reuse.

2 Architecture and acceptance

Lean 4 combines an extensible elaborator with a kernel that checks proof terms in dependent type theory [1]. Lean2LLM uses those interfaces to restrict the role of the proposer. The proposer returns a tactic sequence. Lean executes that sequence to construct a term for a proposition fixed by the caller.

2.1 Fixed request and bounded proposal loop

A request contains a name, a context, and a statement. The context and statement remain fixed throughout a run. The native command `llm_theorem` parses the statement file as one complete Lean term and constructs the theorem syntax structurally. This prevents a statement from closing one declaration and introducing a different target through source concatenation. The native `llm` tactic parses its input as a complete tactic sequence.

When no proposal file is supplied, the tactic emits a structured request with Lean's actual goal. The external driver then invokes a proposer. Each retry receives that goal, the fixed request, the previous candidate, Lean diagnostics, an attempt number, and the remaining time. The driver bounds the attempt count, individual Lean process time, source size, proof size, and memory, and checks an elapsed search budget between calls. Toolchain discovery and module setup have separate limits. A custom proposer is responsible for respecting its deadline; the supplied CLI adapter enforces a subprocess deadline.

Fixed context and proposition ⟶ Lean goal extraction ⟶ candidate proposal ⟶ Lean elaboration and kernel check ⟶ axiom audit ⟶ explicit-source recheck ⟶ stored proof.
Failed candidates return compiler diagnostics to the bounded proposal loop. Retrieved candidates enter at the same checking stage.

The result status `proved` means this run completed the configured acceptance procedure. The status `unknown` covers exhausted attempts, timeouts, missing toolchains, failed elaboration, rejected assumptions, and proposer failures. It carries diagnostics and counters so that a user can distinguish these cases. An `unknown` result establishes no negative mathematical claim.

2.2 Exact theorem and assumptions

The command `audit_llm` resolves the designated theorem in the kernel-checked environment, verifies that the declaration is closed, and rejects unresolved metavariables and proof placeholders. It traverses transitive axiom dependencies. The inherited policy permits `propext`, `Classical.choice`, and `Quot.sound`. These assumptions are reported as part of the checking profile.

The Python driver additionally requires a successful Lean process exit and the expected audit marker. Acceptance also checks the generated explicit source file, independently of the externally supplied tactic-file mechanism. This second pass guards against discrepancies between the accepted candidate and the exported theorem that users will receive. A lexical filter rejects a conservative set of commands and metaprogramming constructs before execution. The native axiom audit supplies a separate logical check: filtering text alone cannot determine all assumptions in a proof's dependency closure.

2.3 Execution boundary

The inherited local runner uses macOS `sandbox-exec`. Checking processes have no network permission or filesystem writes, and readable content is restricted to the temporary inputs, pinned toolchain, trusted modules, and selected runtime paths. This implementation fails closed when the required isolation facility is unavailable. Its recorded tests include actual denied canary reads and writes. Network restrictions are configured; the inherited repository reports no independent penetration test of them.

2.4 Browser workbench

The web workbench uses a separate core Lean WebAssembly build in a fresh isolated worker for each check. Its runtime wrapper is vendored from Browser Lean commit `a8aed095`; the compiler identifies Lean commit `62b6a229`. Runtime asset hashes and license notices accompany the source [6]. The initial compressed core payload is approximately 47.6 MiB. The deployed profile excludes optional Mathlib and other extension layers. It bounds source size and checking time; cancellation disposes of the worker. Browser HTTP caching can reuse downloaded runtime assets across checks. Each submission starts a new compiler worker, keeping the compiler's execution state isolated. Local patches to the pinned runtime propagate compiler errors and aborts to the application. A runtime fault permits one retry in another fresh worker within the overall checking budget. Every completed or cancelled check disposes of its worker.

The browser assembles a restricted theorem source and requires successful Lean completion and a matching axiom report for the requested theorem. Its guards reject placeholders, unsupported commands, and custom

metaprogramming. The browser profile has its own toolchain pin and checker fingerprint, separate from the native bridge. A successful browser check is a local observation. Records can be saved on the device, searched, and exchanged as JSON with the Python library. Imported verification status is reset. A subsequent native check resolves a native fingerprint and creates its corresponding content identity. The site also supports an optional configured account-based storage adapter. Remote server execution would require its own restricted worker and resource policy; the current compiler executes in the browser.

3 Two layers of proof reuse

3.1 Exact requests and a rechecked execution cache

The original cache key hashes the normalized request together with a checker fingerprint. That fingerprint includes the Lean version, executable bytes, native bridge source and compiled module, and Python checking policy. Paths also enter the original fingerprint, making it conservative and specific to the local installation. A changed goal or checker identity selects a different entry. This cache is appropriate for repeating the same experiment.

On a hit, the driver reads the stored request and proof, validates the key and request, reruns Lean and the axiom audit, and rechecks the explicit export. Damaged exports can be reconstructed from a freshly checked stored proof. Consequently a cache hit saves proposal work; it still pays verification cost. The cache is a performance aid under a fixed checking environment.

3.2 Content-addressed records and indexed retrieval

The proof library separates the identity of an artifact from its local verification status. It stores records in SQLite and indexes title, statement, and tags with full-text search. The record identifier is a SHA-256 digest of canonical content. The shared JSON format requires valid Unicode and lossless integer numbers within $\pm(2^{53}-1)$; fractional values and larger numbers travel as strings. Python floats, including `1.0`, are rejected before hashing. The schema version forms part of the exchange format. The content contains the request, tactic body, exported source, backend name, toolchain pin and fingerprint, tags, and provenance metadata.

The verification object is local evidence associated with that content. It contains a status, scope, checking time, diagnostics, and checker fingerprint. Import discards incoming verification claims. New content starts with an unverified local status. When identical content already has a local result, import preserves that existing evidence. Every subsequent reuse still requires a fresh check under the receiving environment's selected profile.

Field	Purpose
Schema and digest	Version the exchange format and identify canonical content.
Request	Preserve the exact context, proposition, and target name.
Proof and source	Retain both proposed tactics and an ordinary Lean export.
Toolchain	Record the pin and checker/dependency fingerprint.
Metadata	Carry title, tags, origin, licenses, and related artifact references.
Verification	Record who or what checked which scope, when, and with which profile.

Table 1: Logical content and verification evidence have separate roles in a portable proof record. A content hash establishes byte-level identity under the serialization rule; a Lean check establishes acceptance in an environment.

Search returns candidates. An exact replay query additionally matches the statement, context, and toolchain identity. Broader queries may find a theorem with useful quantified hypotheses or a lemma that can help prove another goal. The retrieved theorem must be available in the target environment, and Lean must check its actual application. Neither a shared title nor nearby embedding coordinates establish applicability.

An artifact can be general in the ordinary mathematical sense: a theorem quantified over a type, a function, and a natural number applies to every instantiation satisfying its premises. Reusing that theorem requires checking those premises. Automated discovery of useful substitutions, equivalences, or isomorphisms can be layered above retrieval as a proposal mechanism. Its output must include a checked application or transport proof. The storage index provides no general complexity bound for that search.

3.3 Portable bundles and identity

A portable source bundle contains the proof record, its source file, an integrity manifest, pinned bridge and Lake configuration, the checking driver, and a verification entry point. After unpacking, a reader can run the entry point with Python and the required local Lean toolchain without installing the library package. Import validates digests, archive paths, sizes, and symbolic link restrictions before storing the artifact. It does not execute bundled code. The canonical digest covers the content being addressed; verification metadata is excluded from the identity so that a later check can update local status without pretending the mathematical content changed.

A complete dependency identity needs more than a display string such as "Lean 4." The exact toolchain pin, the bridge and checker policy, and the dependency lock manifest determine a verification profile. For richer projects, the dependency closure should include package revisions, imported compiled artifacts, relevant build options, and trusted plugins. A fingerprint is only as complete as the inputs the implementation actually includes. The current restricted Lean/Std bridge has a smaller dependency surface than a Mathlib project; these profiles should be reported separately.

3.4 Python interface

The distribution is named `lean2llm`, with the import namespace `lean_llm`. Storage and search use Python's standard library, including SQLite. Native checking additionally requires the pinned Lean environment and the inherited macOS isolation facility. The following example stores supplied tactics, checks them, searches the catalog, and exports a portable bundle:

```
from lean_llm import ProofStore

with ProofStore("proofs.sqlite3") as store:
    record = store.add(
        {"name": "truth", "context": "", "statement": "True"},
        "exact True.intro", title="Truth example", tags=["example"])
    checked = store.check(record["id"])
    assert checked["check"]["accepted"]
    candidates = store.search("truth")
    store.export_bundle(checked["record"]["id"], "truth.zip")
```

Adding, importing, searching, and checking supplied proofs use no model calls. The `solve` API takes an explicit proposer and an optional store. Its reuse path searches an exact compatible request, rechecks the candidate, and returns the resulting local record. A fresh proposal run uses the same acceptance path.

4 Standalone certificates and statement alignment

The public `navier-stokes-proof` repository provides a concrete portability case study [5]. Its standalone package contains compressed kernel declarations, a pinned challenge, a small checker based on Lean and Std, licenses, provenance, hashes, and a verification entry point. The checker compares designated theorem statements and their referenced definitions against the supplied challenge, audits permitted assumptions, and replays the solution declarations through Lean's kernel. Required mathematical dependencies travel inside the certificate data, so replay avoids a separate Mathlib installation.

This profile differs from submitting a tactic sequence. The bundle pins Lean 4.34.0-rc2, while the initial bridge pins 4.33.1. The library therefore treats certificate replay as an explicit backend with its own verifier identity. A user supplies the trusted local verification program, its expected SHA-256 digest, and the bundle root; imported metadata cannot select an executable. The adapter distinguishes `integrity-checked` from `verifier-succeeded`; the latter records successful execution of the explicit verifier, whose actual theorem-checking scope remains documented. A source export and a kernel certificate can share catalog and provenance conventions while retaining different check procedures and resource budgets.

Formal acceptance also leaves a statement-alignment task. The certificate repository includes a written comparison between the exact formal endpoints and the corresponding informal Navier–Stokes alternatives. Its sources and two AI review reports are preserved separately. The written translation is a conventional argument, and its own status differs from a compiled theorem. Lean2LLM adopts this separation in its export template: a package records the formal statement, its assumptions, an informal description, the direction of the claimed implication, unresolved obligations, and the scope of any review. That structure

helps a future reader distinguish a checked formal result from the additional interpretation required to use it in another setting.

The reusable template follows five stages: preserve the source and licenses; pin the formal target and dependencies; export the proof and recursive dependencies; run the designated verifier and retain its output; document the relationship between the formal target and the intended claim. A small source bundle may require only the first, second, fourth, and fifth stages. The certificate format serves cases where a standalone dependency closure is available and worth its additional storage cost.

5 Evidence, evaluation, and reproduction

5.1 Inherited public results

Table 2 distinguishes evidence published with the two predecessor repositories. These are reported historical runs, inspected for this manuscript. Their timings depend on the local machine, system load, and toolchain. These records supply regression examples and provenance for the implementation.

Artifact	Recorded result	Scope
Initial bridge tests	29 passing tests; 186.933 seconds in the committed log	Orchestration and real Lean checks
Induction demonstration	One live model proposal; three Lean checks; approximately 14 seconds	One quantified theorem
Standalone certificate	94,719 solution declarations replayed; 580.518 seconds	Two pinned formal endpoints

Table 2: Inherited evidence, with distinct checking scopes. The bridge source revision is `a409e42`; the inspected certificate repository revision is `3b01518`. The certificate replay was recorded on 12 September 2026.

The bridge tests exercise accepted and rejected candidates, goal extraction, compiler feedback before retry, time budgets, exact-cache reuse, corrupted cache records, statement injection, direct transitive axiom auditing, and sandbox denial of file operations. They use deterministic candidates with the real Lean compiler. The live demonstration proves a recurrence property for an arbitrary involution: if $f(f(x)) = x$ for every x, then applying f twice per round returns x for every natural-number round count. Induction proves that statement. This result illustrates a reusable quantified theorem whose hypothesis remains explicit.

The certificate run is evidence from a different repository and checker profile. Its declaration count measures replayed proof data. It does not count independently generated theorems, and it is unrelated to the bridge's proof search success rate. The present project packages the storage and verification workflow; the source repository attributes the underlying mathematical formalization to its upstream authors.

5.2 Evaluation of the library layer

The installed `lean2llm==0.2.0` wheel passed 30 tests in a local 35.274-second run: 22 storage, certificate-adapter, and integrity tests, plus eight native Lean integration tests. The run used a clean virtual environment, Python's isolated execution mode, and a working directory outside the source repository. It made zero model calls. The storage cases cover deterministic identities, exact matching, full-text retrieval, bundle round trips, corruption and unsafe archive rejection, and reset of imported verification status. The native cases check fixed-statement acceptance, rejection of unsuitable tactics and placeholders, transitive custom-axiom rejection, and fresh native checking during reuse. They also check that a failed recheck clears a prior local status. An end-to-end case imports a false proof with a forged successful verification label, resets that label, and observes rejection by the real Lean checker. A source bundle was unpacked and verified using isolated Python execution outside the installed package. The additional certificate case rejects a toolchain-file reference that escapes through a symbolic link. These are new implementation results, separate from Table 2.

The final audit also exercised the shared numeric and Unicode boundary and rejected contradictory or wrongly targeted browser axiom reports. Browser verification requires exactly one matching report for its private qualified target. The frontend suite passed 19 tests; its scope includes source guards, record import, checker-output interpretation, and mocked worker lifecycle, runtime-fault recovery, and the one-retry bound.

A manual Chrome regression ran three checks in the same page without a reload: the bundled paired-steps induction theorem was accepted, a tactic offering a proof of `True` for the proposition `False` was rejected with a type mismatch, and the restored induction theorem was accepted. The retained report identifies these three inputs and their outcomes.

A separate interoperability run imported a browser-generated record, including Unicode metadata, into Python. Its incoming successful status was reset to `unverified`. Two real native Lean checks accepted the supplied proof and produced a native verification record with a new toolchain fingerprint and content identifier. A third report inspected the actual public standalone Navier–Stokes template: nine manifest and referenced-file entries passed integrity checking. That inspection did not execute the standalone verifier or repeat the 94,719-declaration kernel replay.

The deterministic suite evaluates these mechanisms within its test cases. Broad model accuracy requires a separately specified theorem set, budget, baseline, and repeated-run protocol. Search ranking quality and large-catalog latency remain subjects for further measurement. No comparative throughput or proof-discovery success rate is inferred from the storage tests.

5.3 Reproduction procedure

A reproducible run begins from a pinned source revision and the version in `lean-toolchain`. The inherited source-only regression commands are:

```
lake build Llm
python3 -m unittest test_bridge -v
lake env lean examples/Verified.lean
```

The request appears in `examples/involution.json`; its accepted theorem is in `examples/Verified.lean`. A reader can check the supplied proof without invoking a model. A fresh model run requires the configured proposer and its own credential mechanism; quotas and model configuration belong in that run's provenance.

The installed-wheel regression log, run record, and companion interoperability and integrity reports reside together:

```
examples/public_platform/
  installed-wheel-tests.txt
  installed-wheel-result.json
  browser-interop-result.json
  navier-template-integrity-result.json
```

The test run used Python 3.13.3, the pinned Lean 4.33.1 distribution, and the native-test opt-in flag. With the wheel installed in a clean environment and the toolchain discoverable, the command has this form:

```
LEAN_LLM_NATIVE_TESTS=1 \
  /path/to/venv/bin/python -I -m unittest discover \
  -s /path/to/repo/tests -v
```

Library reproduction additionally preserves the schema version, canonical record, exported bundle, dependency profile, and local verification report. The artifact identifier supports integrity checks and deduplication. The verification report establishes what was run and the observed outcome. A downloadable proof should provide both. The paper source and submission metadata are included alongside the implementation so that changes to system behavior can be reflected in the accompanying description.

6 Related work and limitations

LeanDojo exposes Lean proof environments and premise information; its ReProver uses retrieval to select premises for generated proofs [2]. Lean Copilot integrates language-model inference into Lean to suggest proof steps, complete goals, and retrieve premises [3]. These systems establish that model proposals, Lean feedback, and retrieval are existing approaches. Lean2LLM emphasizes a small externally orchestrated loop and the subsequent artifact lifecycle: exact replay, indexed storage, portable exchange, and explicit local rechecking. Its paper makes no priority claim for neural theorem proving or retrieval-augmented proof search.

The initial proposer can fail on true theorems, repeatedly generate unsuitable tactics, or spend its entire budget on an unproductive approach. Caching reduces repeated proposal cost when an applicable proof

already exists. A searchable catalog enlarges the set of discoverable candidates. Neither mechanism yields a completeness theorem or a polynomial bound for arbitrary proof search.

The Python driver, database, and web orchestration are covered by executable tests and code review; they have no complete formal correctness proof. The acceptance claim relies on Lean's checking implementation, the selected toolchain and dependencies, the operating system, and correct association of a checked theorem with the requested target. Hashes detect modification relative to a trusted digest; an attacker who controls the reference digest can change both content and manifest. Exporters, archive parsers, and certificate readers remain software components requiring careful validation.

Axiom policies must be visible and versioned. A proof relying on a premise in its statement establishes an implication under that premise. A user importing a theorem also needs its definitions and scope. A successful formal check does not supply missing empirical premises or an omitted correspondence to an informal claim. These boundaries motivate preserving source, dependencies, statement alignment, and verification evidence together.

7 Conclusion

Lean2LLM organizes a proof's lifecycle around a fixed statement and a repeatable checking procedure. The original proposal loop produces Lean source under explicit budgets and assumptions. Exact-cache replay rechecks that source. The proof library makes artifacts addressable, searchable, exportable, and eligible for a fresh local check. A certificate profile extends the same cataloging principles to standalone kernel data. The resulting workflow gives researchers a concrete way to keep, find, inspect, and reuse proofs after the interaction that first produced them.

AI assistance and responsibility. OpenAI Codex assisted with software implementation, source inspection, test construction, literature lookup, and drafting this manuscript. A language model proposed the inherited induction proof; Lean checked the accepted proof term. The certificate repository separately records its own AI review provenance. Octavia Sulea is the author and retains responsibility for review, attribution, and any decision to release or submit this manuscript. Automated checks and AI reviews are described according to their actual scope.

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language (System Description). In *Automated Deduction – CADE 28*, 2021, pp. 625–635. https://doi.org/10.1007/978-3-030-79876-5_37.

[2] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *Advances in Neural Information Processing Systems*, 36, 2023. https://arxiv.org/abs/2306.15626.

[3] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean. arXiv:2404.12534, version 3, 2025. https://arxiv.org/abs/2404.12534v3.

[4] Octavia Sulea. Lean–LLM: local proposal, checking, and cache bridge. Source, examples, and recorded test output, revision `a409e42c806426f6f26ad76787fcebf902898d78`, 2026. https://github.com/mary-octavia/lean-llm/tree/a409e42c806426f6f26ad76787fcebf902898d78.

[5] Octavia Sulea. Navier–Stokes proof: standalone certificate checking and statement alignment. Reproducibility artifacts, revision `3b01518b6e33ab99b36a582219aa5c605be5268a`, 2026. https://github.com/mary-octavia/navier-stokes-proof/tree/3b01518b6e33ab99b36a582219aa5c605be5268a.

[6] Browser Lean contributors. Browser Lean: a Lean WebAssembly runtime and browser integration. Vendored source revision `a8aed095534e67061e2626b94e8e69193099091d`; runtime provenance and asset hashes preserved in the Lean2LLM source tree, 2026. https://github.com/live-codes/browser-lean/tree/a8aed095534e67061e2626b94e8e69193099091d.